Creating a Counter App with useState

In this lesson, we create a tiny counter app with useState in addition to createContext and useContext.

WE'LL COVER THE FOLLOWING

- Before you begin
- What we create
- The code
- The working example
- Exercise
- Next

Before you begin

If you haven't learned React basics and React Hooks basics, please visit the official site before continuing. This course assumes a basic understanding of React and React Hooks.

What we create

We are creating a so-called counter app. There are two numbers that we can increment or decrement. We don't structure code and put everything into one file in this lesson.

The code

```
import React, { createContext, useContext, useState } from 'react';
```

This imports `React` and some hooks from the library.

```
const initialState = {
  count1: 0,
  count2: 0,
```

```
};
```

This defines `initialState` which includes two numbers, `count1` and `count2`

```
const useValue = () => useState(initialState);
```

This is a handy custom hook that we can use in a provider below.

```
const Context = createContext(null);
```

This creates a context. Its value is initially `null`, but will be replaced by `useValue()`.

```
const useGlobalState = () => {
  const value = useContext(Context);
  if (value === null) throw new Error('Please add GlobalStateProvider');
  return value;
};
```

This is a custom hook to return the context value. It checks whether a provider is used as expected. If the value is `null`, it is not updated because `useValue()` never returns `null`.

```
const GlobalStateProvider = ({ children }) => (
  <Context.Provider value={useValue()}>{children}</Context.Provider>
);
```

This is a context provider. This is a component that should be put near the root.

```
const Counter = ({ name }) => {
  const [state, setState] = useGlobalState();
  const count = state[name] || 0;
  const increment = () => {
    setState({ ...state, [name]: count + 1 });
  };
  const decrement = () => {
    setState({ ...state, [name]: count - 1 });
  };
  return (
    <div>
      {count}
```

```
      <button onClick={increment}>+1</button>
      <button onClick={decrement}>-1</button>

    </div>
  );
};
```

This is a component to display a number. It receives a `name` prop to specify which number to display, `count1` or `count2`. It has two buttons to increment and decrement the number.

```
const App = () => (
  <GlobalStateProvider>
    <h1>Count1</h1>
    <Counter name="count1" />
    <Counter name="count1" />
    <h1>Count2</h1>
    <Counter name="count2" />
    <Counter name="count2" />
  </GlobalStateProvider>
);
```

This is an app component. It has `GlobalStateProvider` and several `Counter` components.

```
export default App;
```

Finally, we export the app component.

The working example

Check out the app below.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Exercise

Try adding a new counter, `count3`. Interestingly, you don't need to modify `initialState` if the initial value of `count3` is `0`.

Next

In the next lesson, we will learn a different pattern with `useReducer`.

Creating a Counter App with useReducer

In this lesson, we create a tiny counter app with useReducer instead of useState, as in the previous lesson.

WE'LL COVER THE FOLLOWING

- Why useReducer
- The code
- The working example
- Exercise
- Notes for Redux users
- Next

Why useReducer

The `useState` hook is fine for most cases. The `useReducer` hook is a variant of `useState` which allows updating logic separately; it is also known as a **reducer**.

While it's a preference whether or not you choose to use the *reducer*, it's still good to learn the difference.

The code

```
import React, { createContext, useContext, useReducer } from 'react';
```

We import `useReducer` instead of `useState`.

```
const initialState = {
  count1: 0,
  count2: 0,
};
```

This is the same as the previous example. Note: If we add `count3`, we'd want

to add it in the `initialState` too, although this is not necessary.

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return {
        ...state,
        [action.name]: state[action.name] + 1,
      };
    case 'DECREMENT':
      return {
        ...state,
        [action.name]: state[action.name] - 1,
      };
    default:
      return state;
  }
};
```

This defines state updating logic. This pattern is popular in Redux.

Note that using `action.name` for the state property might not be a recommended pattern. It would be better to be more specific in reducers.

```
const useValue = () => useReducer(reducer, initialState);
```

This is a handy custom hook that we can use in the provider below.

```
const Context = createContext(null);

const useGlobalState = () => {
  const value = useContext(Context);
  if (value === null) throw new Error('Please add GlobalStateProvider');
  return value;
};

const GlobalStateProvider = ({ children }) => (
  <Context.Provider value={useValue()}>{children}</Context.Provider>
);
```

The code above is the same as in the previous lesson.

```
const Counter = ({ name }) => {
  const [state, dispatch] = useGlobalState();
```

```
  const [state, dispatch] = useGlobalState();
  return (
    <div>
      {state[name]}
      <button onClick={() => dispatch({ type: 'INCREMENT', name })}>+1</bu
tton>
      <button onClick={() => dispatch({ type: 'DECREMENT', name })}>-1</bu
tton>
    </div>
  );
};
```

In this component, we use `dispatch` instead of `setState`.

```
const App = () => (
  <GlobalStateProvider>
    <h1>Count1</h1>
    <Counter name="count1" />
    <Counter name="count1" />
    <h1>Count2</h1>
    <Counter name="count2" />
    <Counter name="count2" />
  </GlobalStateProvider>
);

export default App;
```

The code for the app component is the same as in the previous lesson.

The working example

Check out the app below.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Exercise

How would you add a double button “x2”?

Notes for Redux users

If you are familiar with Redux, you might wonder why we aren't using action creators. In this lesson, we prefer simpler dispatch. Even in practical apps, we wouldn't need action creators if we had a type system like TypeScript.

Next

In the next chapter, we will introduce React Tracked and learn about render optimization.

Understanding Render Optimization

In this lesson, we will learn what render optimization is and when we need to use it.

WE'LL COVER THE FOLLOWING

- Visualizing renders
- Check the app
- Render optimization
- The primary approach to render optimization
- Render optimization by React Tracked
- Next

Visualizing renders

Before explaining render optimization, we must modify our counter app so that we can visualize renders.

```
import { useRef, useEffect } from 'react';

const useFlasher = () => {
  const ref = useRef(null);
  useEffect(() => {
    ref.current.classList.add('flash');
    setTimeout(() => {
      ref.current.classList.remove('flash');
    }, 300);
  });
  return ref;
};
```

This adds `flash` class to a DOM that the hook is attached to.

We assume that `flash` class style is defined in the css.

```
const Counter = ({ name }) => {
  const [state, dispatch] = useGlobalState();

  return (
    <div ref={useFlasher()}>
      {state[name]}
      <button onClick={() => dispatch({ type: 'INCREMENT', name })}>+1</bu
tton>
      <button onClick={() => dispatch({ type: 'DECREMENT', name })}>-1</bu
tton>
    </div>
  );
};
```

The counter component from the previous example is modified just a bit. The only change is `ref={useFlasher()}`.

Check the app

Now, let's see how it behaves in the following app.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Render optimization

The app above should behave in a way that all components (the 4 numbers) render if any button is clicked. However, **Count2** components wouldn't need to re-render when only `count1` is incremented or decremented, and vice versa.

Render optimization means only those components re-render that display a change made to the previous version. This is important for performance in certain cases.

The primary approach to render optimization

In most cases, render optimization is not necessary as long as the app works.

In some intensive apps, performance is critical.

The primary approach in such apps is to split a context into multiple contexts. In our counter app, we can create two contexts for two counts.

Render optimization by React Tracked

For certain reasons, we might not want to split a context; maybe because the state is tightly coupled and it makes sense to update the state at once.

React Tracked solves this issue by state usage tracking. State usage tracking is to track the usage of the state in render and only trigger re-renders if the used part of the state is changed. In other words, if you click “+1” in Count1, only Count1 components re-render, and Count2 components will not re-render.

Next

We will see how render optimization behaves with React Tracked in the next lesson.

Modifying the Counter App with React Tracked

In this lesson, we use React Tracked in the counter app and see how it behaves.

WE'LL COVER THE FOLLOWING

- Installing React Tracked
- Modify code
- Check the app
- Behavior with React Tracked
- Technique behind React Tracked
- Next

Installing React Tracked

In this course, the platform already has React Tracked installed.

If you run the code locally, please follow the instructions here.

Modify code

Previously, we had the following code for our counter app:

```
const Context = createContext(null);

const useGlobalState = () => {
  const value = useContext(Context);
  if (value === null) throw new Error('Please add GlobalStateProvider');
  return value;
};

const GlobalStateProvider = ({ children }) => (
  <Context.Provider value={useValue()}>{children}</Context.Provider>
);
```

This creates a context, a hook, and a provider.

In React Tracked, this is done by `createContainer`.

```
import { createContainer } from 'react-tracked';

const {
  useTracked: useGlobalState,
  Provider: GlobalStateProvider,
} = createContainer(useValue);
```

Notice we simply rename a hook and a provider from the default names in a container.

Check the app

Now, let's try the modified app.

```
import React from 'react';
require('./style.css');

import ReactDOM from 'react-dom';
import App from './app.js';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

Behavior with React Tracked

If you click the "+1" button in the app above, you'll notice that only two components re-render. If `count1` is changed, Count1 components are flashed, and if `count2` is changed, Count2 components are flashed.

Renders are optimized based on the state usage in components. This is effortless. The only change we made was to create a container using the library, no changes were made to the components and reducer.

Technique behind React Tracked

The library utilizes Proxy. It tracks how a state object property is accessed in render and marks it as "used." If the used property is changed by updates, it will trigger re-render. The use of Proxy is not new in JavaScript frontend

libraries, but tracking the usage in React render functions is unique to this library.

Next

In the next chapter, we will learn how to make a more realistic ToDo app.

Initial State and Reducer

In this lesson, we will define the initialState and reducer for a simple Todo app with useReducer.

WE'LL COVER THE FOLLOWING

- Initial state
- Reducer
- Store file
- Next

Initial state

We are developing a ToDo app. Our state has a list of ToDo items. Let's put three items initially so that we see them when the app starts.

```
const initialState = {
  todos: [
    { id: 1, title: 'Wash dishes' },
    { id: 2, title: 'Study JS' },
    { id: 3, title: 'Buy ticket' },
  ],
  query: '',
};
```

The IDs are almost always recommended when we have a list in a state. We use them to specify the `key` prop in React.

Right now, an item has only `title` in addition to `id`. We will be using another property, `completed`, that is undefined initially.

There is also `query` which we will use to highlight ToDo titles.

Now, let's define a reducer function to manipulate this state immutably.

Reducer

Reducer

A **reducer** is a function to update a state with an action. An action is an object, typically with a `type` property. A reducer returns a new state without mutating any objects in it.

As we need to keep track of `nextId`, we define it in advance as an easy solution.

```
let nextId = 4;
```

The solution is `4` because we have three items initially.

We then define our reducer.

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'ADD_TODO':
      return {
        ...state,
        todos: [...state.todos, { id: nextId++, title: action.title }],
      };
    case 'DELETE_TODO':
      return {
        ...state,
        todos: state.todos.filter(todo => todo.id !== action.id),
      };
    case 'TOGGLE_TODO':
      return {
        ...state,
        todos: state.todos.map(todo =>
          todo.id === action.id
            ? { ...todo, completed: !todo.completed }
            : todo,
        ),
      };
    case 'SET_QUERY':
      return {
        ...state,
        query: action.query,
      };
    default:
      return state;
  }
```

```
};
```

It is a bit long, but there is a pattern. For each action type, it creates a new state. Let's summarize what each action type means.

- `ADD_TODO` creates a new item and appends it to the end of the list. It requires the title.
- `DELETE_TODO` removes an existing item from the list. It requires the `id`.
- `TOGGLE_TODO` changes the `completed` flag of the item. It requires the `id`.
- `SET_QUERY` sets `query` to highlight ToDo titles.

Store file

We create a file named `store.js` and export some hooks and a component.

Using `initialState` and `reducer`, the file is as follows:

```
// store.js

import { useReducer } from 'react';
import { createContainer } from 'react-tracked';

const initialState = ...;

let nextId = 4;

const reducer = ...;

const useValue = () => useReducer(reducer, initialState);

export const {
  Provider,
  useTrackedState,
  useUpdate: useDispatch,
} = createContainer(useValue);
```

That's all we need to define our global state.

Next

In the next lesson, we will create a TodoList component.

TodoList Component

In this lesson, we develop a component to display the list of ToDo items.

WE'LL COVER THE FOLLOWING

- TodoList
- Next

TodoList

Here is the code for listing ToDo items.

```
import React from 'react';
import { useDispatch, useTrackedState } from './store';

import TodoItem from './TodoItem';
import NewTodo from './NewTodo';

const TodoList = () => {
  const dispatch = useDispatch();
  const state = useTrackedState();
  const setQuery = event => {
    dispatch({ type: 'SET_QUERY', query: event.target.value });
  };
  return (
    <div>
      <ul>
        {state.todos.map(({ id, title, completed }) => (
          <TodoItem key={id} id={id} title={title} completed={completed} /
>
        ))}
        <NewTodo />
      </ul>
      <div>
        Highlight Query for incomplete items:
        <input value={state.query} onChange={setQuery} />
      </div>
```

```
    </div>
  );
};

export default TodoList;
```

The `useDispatch` returns a function `dispatch`. When we dispatch an action it will invoke the reducer.

The `useTrackedState` returns a global state.

Notice this signature is exactly the same as the usage of `useReducer`.

```
const [state, dispatch] = useReducer(...);
```

We have another hook in React Tracked that can be used like this.

```
const [state, dispatch] = useTracked();
```

There are two components this imports from other files, `TodoItem` and `NewTodo` which we will cover later.

Next

In the next lesson, we create a TodoItem component.

TodoItem Component

In this lesson, we create a TodoItem component that displays one item in the list.

WE'LL COVER THE FOLLOWING

- TodoItem
- Next

TodoItem

We create a TodoItem component, based on several hooks we created: `useQuery`, `useDeleteTodo`, `useToggleTodo`, and `useFlasher`

```
import React from 'react';

import { useDispatch, useTrackedState } from './store';
import { useFlasher } from './utils';

const renderHighlight = (title, query) => {
  if (!query) return title;
  const index = title.indexOf(query);
  if (index === -1) return title;
  return (
    <React.Fragment>
      {title.slice(0, index)}
      <b>{query}</b>
      {title.slice(index + query.length)}
    </React.Fragment>
  );
};

const TodoItem = ({ id, title, completed }) => {
  const dispatch = useDispatch();
  const state = useTrackedState();
  const delTodo = () => {
    dispatch({ type: 'DELETE_TODO', id });
```

```
  };
  return (
    <li ref={useFlasher()}>
      <input
        type="checkbox"
        checked={!!completed}
        onChange={() => dispatch({ type: 'TOGGLE_TODO', id })}
      />
      <span
        style={{
          textDecoration: completed ? 'line-through' : 'none',
        }}
      >
        {completed ? title : renderHighlight(title, state.query)}
      </span>
      <button onClick={delTodo}>Delete</button>
    </li>
  );
};

export default React.memo(TodoItem);
```

This component receives 3 props: `id`, `title`, and `completed`. They are all primitive values. An alternative way is to receive 1 prop, `todo` object.

This is specific to React Tracked, but there's a pitfall with object props if you use React.memo. With React Tracked, we recommend primitive props for components that are wrapped by `React.memo`. If one needs object props there's a workaround, but it's out of the scope of this course.

Next

In the next lesson, we create `NewTodo` component.

NewTodo Component

In this lesson, we learn how to develop a NewTodo component.

WE'LL COVER THE FOLLOWING

- NewTodo
- Next

NewTodo

```
import React, { useState } from 'react';

import { useDispatch } from './store';
import { useFlasher } from './utils';

const NewTodo = () => {
  const dispatch = useDispatch();
  const [text, setText] = useState('');
  const addTodo = () => {
    dispatch({ type: 'ADD_TODO', title: text });
    setText('');
  };
  return (
    <li ref={useFlasher()}>
      <input
        value={text}
        placeholder="Enter title..."
        onChange={e => setText(e.target.value)}
      />
      <button onClick={addTodo}>Add</button>
    </li>
  );
};

export default React.memo(NewTodo);
```

This component has a local state `text` for a text field. The local state is

This component has a local state, `text`, for a text field. The local state is created with the React basic hook `useStat`. Only when a button is clicked, will

it dispatch an action to the store.

Next

In the next lesson, we finish up our app.

App Component and File Structure

In this lesson, we develop and list all files.

WE'LL COVER THE FOLLOWING

- App
- Utils
- File structure
- Next

App

```
import React from 'react';

import { Provider } from './store';
import TodoList from './TodoList';

const App = () => (
  <Provider>
    <TodoList />
  </Provider>
);

export default App;
```

This is the root component that wraps `TodoList` in `Provider`.

Utils

We have one more file that exports `useFlasher` used in the previous lesson.

```
import { useEffect, useRef } from 'react';

export const useFlasher = () => {
  const ref = useRef(null);
```

```
  useEffect(() => {
    ref.current.classList.add('flash');

    setTimeout(() => {
      ref.current.classList.remove('flash');
    }, 300);
  });
  return ref;
};
```

File structure

As we have only a small number of files, we put them all flat.

- App.js
- store.js
- TodoList.js
- TodoItem.js
- NewTodo.js
- utils.js

Files with “PascalCase” names are components.

Next

In the next lesson, we check the app behavior with a working demo.

Demo

In this lesson, we demonstrate the ToDo app.

WE'LL COVER THE FOLLOWING

- Try demo
- How to confirm render optimization
- Next

Try demo

We simply put together all the code from previous lessons. Run it, and see how it works.

```
import { useReducer } from 'react';
import { createContainer } from 'react-tracked';

const initialState = {
  todos: [
    { id: 1, title: 'Wash dishes' },
    { id: 2, title: 'Study JS' },
    { id: 3, title: 'Buy ticket' },
  ],
  query: '',
};

let nextId = 4;

const reducer = (state, action) => {
  switch (action.type) {
    case 'ADD_TODO':
      return {
        ...state,
        todos: [...state.todos, { id: nextId++, title: action.title }],
      };
    case 'DELETE_TODO':
      return {
        ...state,
        todos: state.todos.filter(todo => todo.id !== action.id),
      };
    case 'TOGGLE_TODO':
      return {
        ...state,
```

```
        todos: state.todos.map(todo =>
          todo.id === action.id
            ? { ...todo, completed: !todo.completed }
            : todo,
        ),
      };
    case 'SET_QUERY':
      return {
        ...state,
        query: action.query,
      };
    default:
      return state;
  }
};

const useValue = () => useReducer(reducer, initialState);

export const {
  Provider,
  useTrackedState,
  useUpdate: useDispatch,
} = createContainer(useValue);
```

How to confirm render optimization

The highlight query is only effective against incomplete ToDo items. This means that complete items do not update with the query change.

You can try it with the following steps.

1. Click checkboxes of some items that will make those items complete. Complete items are shown with a line through them.
2. Type some characters in the query field at the bottom.
3. You will see only incomplete items (not shown with a line through) re-render (become bold).

Next

In the next chapter, we create the same ToDo app in a more structured way.

Simple Store with useState

In this lesson, we learn a different pattern with useState and custom hooks, create a store in a couple of lines, and also make use of Immer.

WE'LL COVER THE FOLLOWING

- Initial state
- useValue
- createContainer
- useSetDraft
- The entire store file
- Next

Initial state

The initial state is the same as in the previous lessons.

```
const initialState = {
  todos: [
    { id: 1, title: 'Wash dishes' },
    { id: 2, title: 'Study JS' },
    { id: 3, title: 'Buy ticket' },
  ],
  query: '',
};
```

useValue

The initial state is only needed for `useState`.

```
const useValue = () => useState(initialState);
```

createContainer

We create a store with `createContainer`.

```
const {
  Provider,
  useTrackedState,
  useUpdate: useSetState,
} = createContainer(
  useValue,
);
```

Notice we renamed `useUpdate` to `useSetState`.

useSetDraft

Now, we create a wrapper function on `useSetState`.

```
const useSetDraft = () => {
  const setState = useSetState();
  return useCallback(
    draftUpdater => {
      setState(produce(draftUpdater));
    },
    [setState],
  );
};
```

This `useSetDraft` uses Immer's `produce`. Immer allows a mutating draft state while we get an immutably updated state in the end.

The entire store file

Our `store.js` is simply the concatenation of what we defined above.

```
// store.js

import { useState, useCallback } from 'react';
import { createContainer } from 'react-tracked';
import produce from 'immer';

const initialState = {
  todos: [
    { id: 1, title: 'Wash dishes' },
    { id: 2, title: 'Study JS' },
    { id: 3, title: 'Buy ticket' },
```

```
  ],
  query: '',
};

const useValue = () => useState(initialState);

const { Provider, useTrackedState, useUpdate: useSetState } = createContai
ner(
  useValue,
);

const useSetDraft = () => {
  const setState = useSetState();
  return useCallback(
    draftUpdater => {
      setState(produce(draftUpdater));
    },
    [setState],
  );
};

export { Provider, useTrackedState, useSetDraft };
```

Next

Let's create various custom hooks based on the store.

In the next lesson, we create the first `useTodoList` hook.

useTodoList Hook

In this lesson, we create the hook to get the ToDo list.

WE'LL COVER THE FOLLOWING

- useTodoList
- Next

useTodoList

This hook is to simply return the ToDo list.

```
// ./hooks/useTodoList.js

import { useTrackedState } from '../store';

export const useTodoList = () => {
  const state = useTrackedState();
  return state.todos;
};
```

One might think this doesn't have to be a custom hook, but by hiding the implementation detail, we can refactor this hook easily. For example, we may be able to implement it with `useSelector` instead of `useTrackedState`.

Next

In the next lesson, we will learn another hook `useAddTodo`.

useAddTodo Hook

In this lesson, we create a hook to add a new ToDo item.

WE'LL COVER THE FOLLOWING

- useAddTodo
- Next

useAddTodo

We create a hook that returns a callback function which allows us to add a new ToDo item.

```
// ./hooks/useAddTodo.js

import { useCallback } from 'react';
import { useSetDraft } from '../store';

let nextId = 100;

export const useAddTodo = () => {
  const setDraft = useSetDraft();
  return useCallback(
    title => {
      setDraft(draft => {
        draft.todos.push({ id: nextId++, title });
      });
    },
    [setDraft],
  );
};
```

With `setDraft` we can mutate the draft state. The `nextId` is defined outside of the hook to provide a unique ID every time the callback is called.

We explicitly use `useCallback` for returning the stable callback. It's not

necessary if we are sure that stability is not important. But, we don't know at

this point if the callback function will be passed for props of child components that are wrapped by React.memo. Hence, it's a safe bet to always use `useCallback` in custom hooks.

Next

We continue to create another hook. In the next lesson, we learn to create `useDeleteTodo`.

useDeleteTodo Hook

In this lesson, we create a hook to delete a ToDo item.

WE'LL COVER THE FOLLOWING

- useDeleteTodo
- Next

useDeleteTodo

Similar to the previous lesson, we create a hook that returns a callback that allows us to delete an item.

```
// ./hooks/useDeleteTodo.js

import { useCallback } from 'react';
import { useSetDraft } from '../store';

export const useDeleteTodo = () => {
  const setDraft = useSetDraft();
  return useCallback(
    id => {
      setDraft(draft => {
        const index = draft.todos.findIndex(todo => todo.id === id);
        if (index >= 0) draft.todos.splice(index, 1);
      });
    },
    [setDraft],
  );
};
```

Again, we use Immer's feature here, to show what a mutable update looks like.

We could use an immutable update if we prefer, like the following:

```
setDraft(draft => draft.filter(todo => todo.id !=== id));
```

In this case, the code with the immutable update is shorter.

Next

In the next lesson, we learn about `useToggleTodo` hook.

useToggleTodo Hook

In this lesson, we create a hook to toggle a ToDo item.

WE'LL COVER THE FOLLOWING

- useToggleTodo
- Next

useToggleTodo

This hook is to toggle the `completed` flag of an item.

```
// ./hooks/useToggleTodo.js

import { useCallback } from 'react';
import { useSetDraft } from '../store';

export const useToggleTodo = () => {
  const setDraft = useSetDraft();
  return useCallback(
    id => {
      setDraft(draft => {
        const todo = draft.todos.find(todo => todo.id === id);
        if (todo) todo.completed = !todo.completed;
      });
    },
    [setDraft],
  );
};
```

Initially `todo.completed` is `undefined`, but once it's toggled it will be `true`, and `false` / `true` thereafter.

This hook can also be implemented without mutations. The immutable version is the following:

```
setDraft(draft => draft.map(todo => (todo.id === id ? { ...todo, completed
: !todo.completed } : todo)));
```

This might be a bit tricky if you aren’t used to immutable updates.

Next

In the next lesson, we learn about another hook `useQuery`.

useQuery Hook

In this lesson, we create a hook to handle queries.

WE'LL COVER THE FOLLOWING

- useQuery
- Next

useQuery

A query in our app is to filter ToDo items. This hook returns two functions, `setQuery` and `getQuery`

`setQuery` is a function to update `query`, while `getQuery` is a function to read `query`.

```
// ./hooks/useQuery.js

import { useCallback } from 'react';
import { useTrackedState, useSetDraft } from '../store';
export const useQuery = () => {
  const state = useTrackedState();
  const getQuery = () => state.query;
  const setDraft = useSetDraft();
  const setQuery = useCallback(
    query => {
      setDraft(draft => {
        draft.query = query;
      });
    },
    [setDraft],
  );
  return { getQuery, setQuery };
};
```

There is an important note here: The reason we use `getQuery` is because we

used React Tracked. If we just used the bare React context, we could simply do the following:

```
return {
  query: state.query,
  setQuery,
};
```

If we do this with React Tracked, `query` is always marked as “used” when using this hook because it accesses the `query` property of the state inside the hook.

Another recommended pattern is that `useQuery` only returns `setQuery` and is named `useSetQuery`. Another pattern could be that you use `state.query` directly in components.

Next

In the next lessons, we learn how to create components with the hooks we created.

TodoList Component

In this lesson, we create a component to list ToDo items.

WE'LL COVER THE FOLLOWING

- TodoList
- Next

TodoList

Based on the `useTodoList` hook, we create the TodoList component.

```
// ./components/TodoList.js

import React from 'react';

import { useTodoList } from '../hooks/useTodoList';
import { useQuery } from '../hooks/useQuery';
import TodoItem from './TodoItem';
import NewTodo from './NewTodo';

const TodoList = () => {
  const { getQuery, setQuery } = useQuery();
  const todos = useTodoList();
  return (
    <div>
      <ul>
        {todos.map(({ id, title, completed }) => (
          <TodoItem key={id} id={id} title={title} completed={completed} /
>
        ))}
        <NewTodo />
      </ul>
      <div>
        Highlight Query for incomplete items:
        <input value={getQuery()} onChange={e => setQuery(e.target.value)}
 />
```

```
      </div>
    </div>
  );
};

export default TodoList;
```

We also use the `useQuery` hook to show the text field for the highlight query in this component.

Next

In the next two lessons we create two components, `TodoItem` and `NewTodo`, that are imported into the `TodoList`.

TodoItem Component

In this lesson, we create a component to show one ToDo item.

WE'LL COVER THE FOLLOWING

- TodoItem
- Next

TodoItem

Based on several hooks we created, we can now create the TodoItem component.

```
// ./components/TodoItem.js

import React from 'react';

import { useQuery } from '../hooks/useQuery';
import { useDeleteTodo } from '../hooks/useDeleteTodo';
import { useToggleTodo } from '../hooks/useToggleTodo';
import { useFlasher } from '../utils';

const renderHighlight = (title, query) => {
  if (!query) return title;
  const index = title.indexOf(query);
  if (index === -1) return title;
  return (
    <React.Fragment>
      {title.slice(0, index)}
      <b>{query}</b>
      {title.slice(index + query.length)}
    </React.Fragment>
  );
};

const TodoItem = ({ id, title, completed }) => {
  const { getQuery } = useQuery();
```

```
  const { getQuery } = useQuery();
  const deleteTodo = useDeleteTodo();
  const toggleTodo = useToggleTodo();
  return (
    <li ref={useFlasher()}>
      <input
        type="checkbox"
        checked={!!completed}
        onChange={() => toggleTodo(id)}
      />
      <span
        style={{
          textDecoration: completed ? 'line-through' : 'none',
        }}
      >
        {completed ? title : renderHighlight(title, getQuery())}
      </span>
      <button onClick={() => deleteTodo(id)}>Delete</button>
    </li>
  );
};

export default React.memo(TodoItem);
```

We wrap this component with React.memo when exporting. This allows eliminating re-renders if this item is not changed.

Next

In the next lesson, we create a component `NewTodo` to create a new ToDo item.

NewTodo Component

In this lesson, we create a component to add a new ToDo item.

WE'LL COVER THE FOLLOWING

- NewTodo
- Next

NewTodo

Based on the `useAddTodo` hook, we create a NewTodo component.

```
// ./components/NewTodo.js

import React, { useState } from 'react';

import { useAddTodo } from '../hooks/useAddTodo';
import { useFlasher } from '../utils';

const NewTodo = () => {
  const addTodo = useAddTodo();
  const [text, setText] = useState('');
  return (
    <li ref={useFlasher()}>
      <input
        value={text}
        placeholder="Enter title..."
        onChange={e => setText(e.target.value)}
      />
      <button
        onClick={() => {
          addTodo(text);
          setText('');
        }}
      >
        Add
      </button>
```

```
    </li>
  );
};

export default React.memo(NewTodo);
```

Next

In the next lesson, we will finally create a root component `App`.

App Component and File Structure

In this lesson, we create an App component.

WE'LL COVER THE FOLLOWING

- App
- Utils
- Folder structure
- Next

App

This is the root component that wraps the TodoList in the Provider.

```
// ./components/App.js

import React from 'react';

import { Provider } from './store';
import TodoList from './TodoList';

const App = () => (
  <Provider>
    <TodoList />
  </Provider>
);

export default App;
```

Utils

We have one more file that exports `useFlasher`.

```
// ./utils.js
```

```
import { useEffect, useRef } from 'react';


export const useFlasher = () => {
  const ref = useRef(null);
  useEffect(() => {
    ref.current.classList.add('flash');
    setTimeout(() => {
      ref.current.classList.remove('flash');
    }, 300);
  });
  return ref;
};
```

Folder structure

Our files are stored in different directories. Below is a list of all the files:

```
store.js
hooks/useTodoList.js
hooks/useAddTodo.js
hooks/useDeleteTodo.js
hooks/useToogleTodo.js
hooks/useQuery.js
components/TodoList.js
components/TodoItem.js
components/NewTodo.js
components/App.js
utils.js
```

Next

In the next lesson, we check the app behavior with a working demo.

Demo

In this lesson, we see the running demo.

WE'LL COVER THE FOLLOWING

- Try the demo
- Next

Try the demo

We simply put together all the code from previous lessons. Run it, and see how it works.

```
import React from 'react';

import { Provider } from '../store';
import TodoList from './TodoList';

const App = () => (
  <Provider>
    <TodoList />
  </Provider>
);

export default App;
```

Next

All hands-on lessons are now finished. In the final chapter, we cover some further topics that are worth learning.

Async Functions

In this lesson, we briefly go over the async functions.

WE'LL COVER THE FOLLOWING

- Async functions
- useEffect
- use-reducer-async
- Next

Async functions

Async functions are typically used to fetch data from a server. There are different types of async functions, like timers, but the basics are the same: They don't return the result synchronously. Async functions run outside React render functions and require a special way of coding.

useEffect

Often, useEffect is used to start async functions. That's one valid way, but it tends to work unexpectedly and it will not be mainstream once Suspense for data fetching lands.

use-reducer-async

There are several libraries to start async functions in dispatch or event callback. One of these is use-reducer-async, which works well for small async functions. It can be used with React Tracked.

Next

In the next lesson, we get a brief overview of how the Redux ecosystem can help us.

Redux Ecosystem

In this lesson, we briefly introduce Redux.

WE'LL COVER THE FOLLOWING

- Things we can learn from Redux
- Next

Things we can learn from Redux

Redux is a small library, but its ecosystem is huge. Even though we don't use Redux directly, it helps us. The Redux ecosystem provides various best practices, documents, utility libraries, and so on. React Tracked is influenced by Redux, and lots of knowledge in the Redux ecosystem is applicable to React Tracked.

Check out the Redux Style Guide, for example.

Next

In the next lesson, we show a simpler alternative to React Tracked.

Simpler Alternative

In this lesson, we introduce an alternative library to React Tracked.

WE'LL COVER THE FOLLOWING

- React Context
- An alternative
- Next

React Context

While React Context can be used for global state, it is primitive. That's why there are various libraries around it, and React Tracked is one of them. The strength of React Tracked is Proxy-based tracking, but we don't need this for smaller use cases.

An alternative

Out of various libraries, react-hooks-global-state covers a small use case. It only allows you to specify a part of the global state by object keys and is hardly misused.

Next

Finally, we wrap up our entire course.

Wrap-up

At last, there's one thing to note.

WE'LL COVER THE FOLLOWING

- Trend changes rapidly
- Enjoy coding

Trend changes rapidly

In web development, the trends tend to change rapidly. That's a good thing for improving technology, but it's hard for learners.

Fortunately, the React documents are improved and up to date. My recommendation is to check out the official documentation site.

Enjoy coding

A final word: Enjoy coding. That's it.